CAMIRA Input File Reference Manual

Mark George

Input File Syntax and Structure

All solver setup and configuration is done through the input file, which is passed to the solver executable. The basic input file structure is given below.

```
1  // A comment
2
3  #include "OtherInputSettings.inp"
4
5  Section {
6      parameter1 = 6.63e-34
7      parameter2 = 137
8
9      SubSection {
10         configuration = someOption
11         subparameter1 = "filename.csv"
12         subparameter2 = 1
13     }
14 }
```

The `#include` directive on line 3 allows the input file to be split up into multiple files. The directive essentially copy and pastes the contents of the included file at the location of the directive.

Some essential syntax rules:

- Everything is case sensitive.
- Scientific notation i.e. `5.67e-8` can only be used for parameters which are floating point numbers.
- The file structure is completely free-form, meaning braces can be put anywhere, and indenting is ignored. However, there must be a new line after a parameter is specified with an "`=`".
- Input file options can be put in any order, as long as they follow the correct tree structure.
- White space is ignored, except inside double quotes e.g. `"white space not ignored here"`.

The rest of this documents gives all the required and optional solver parameters that the input file requires. An example of a complete input file is given at the end of this document.

Contents

Part I

CAMIRA-FLOW

1 Model

All physical modelling parameters.

```
Model {
    <model parameters>

    TurbulenceModel {
        <TurbulenceModel parameters>
    }

}
```

Parameters

nu

- **Description**: Constant fluid kinematic viscosity
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `nu = 1.43e-5`

transient

- **Description**: Whether or not the simulation is transient. This dictates if a discrete transient term is included in the equations.
- **Required/Optional**: required
- **Type**: boolean
- **Options**: `yes`, `no`
- **Example**: `transient = yes`

`TurbulenceModel` parameters

Turbulence model parameters only need to be specified if `transient = no`.

model

- **Description**: Type of turbulence model to use.
- **Required/Optional**: Required
- **Type**: Word option.

- **Options**:

laminar	Solves the laminar steady equations.
PrandtlZeroEquation	Prandtl's mixing length model.
ZEQ0	Chen, Q., & Xu, W. (1998). A zero-equation turbulence model for indoor airflow simulation. Energy and buildings, 28(2), 137-144.
ZEQ1	"ZEQ1" from Liu, J., Heidarinejad, M., Pitchurov, G., Zhang, L., & Srebric, J. (2018). An extensive comparison of modified zero-equation, standard $k-\varepsilon$, and LES models in predicting urban airflow. Sustainable cities and society, 40, 28-43.
ZEQ2	"ZEQ2" from Liu, J., Heidarinejad, M., Pitchurov, G., Zhang, L., & Srebric, J. (2018). An extensive comparison of modified zero-equation, standard $k-\varepsilon$, and LES models in predicting urban airflow. Sustainable cities and society, 40, 28-43.
ZEQ3	"ZEQ3" from Liu, J., Heidarinejad, M., Pitchurov, G., Zhang, L., & Srebric, J. (2018). An extensive comparison of modified zero-equation, standard $k-\varepsilon$, and LES models in predicting urban airflow. Sustainable cities and society, 40, 28-43.
ZEQ4	"ZEQ4" from Liu, J., Heidarinejad, M., Pitchurov, G., Zhang, L., & Srebric, J. (2018). An extensive comparison of modified zero-equation, standard $k-\varepsilon$, and LES models in predicting urban airflow. Sustainable cities and society, 40, 28-43.

.

- **Example**: `model = ZEQ3`

Each turbulence model in the table above requires additional parameters that are specified in the same tree, these are given below.

`laminar`

No additional parameters required.

`PrandtlZeroEquation`

No additional parameters required.

ZEQ0

No additional parameters required.

`ZEQ1`

inflowReBuildingHeight

- **Description**: Inflow Reynolds number at average building height. Velocity at average building height is used.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `inflowReBuildingHeight = 183442.2`

inflowTurbIntensityBuildingHeight

- **Description**: Inflow turbulence intensity at average building height. Given as a fraction.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `inflowTurbIntensityBuildingHeight = 0.1`

`ZEQ2`

averageBuildingHeight

- **Description**: Average building height of geometry.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `averageBuildingHeight = 2.2`

inflowVelocityBuildingHeight

- **Description**: Inflow velocity at average building height.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `inflowVelocityBuildingHeight = 5.56`

inflowTKEBuildingHeight

- **Description**: Inflow turbulent kinetic at average building height.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `inflowTKEBuildingHeight = 5.56`

inflowIntegralTimescaleBuildingHeight

- **Description**: Inflow integral timescale at average building height.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `inflowIntegralTimescaleBuildingHeight = 1.5`

roughnessLength

- **Description**: Roughness length z_0 of boundary layer.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `roughnessLenght = 0.004`

heightAxis

- **Description**: Axis which vertical height is measured along.
- **Required/Optional**: required
- **Type**: Named option
- **Options**: `x`, `y`, or `z`
- **Example**: `heightAxis = z`

`ZEQ3`

averageBuildingHeight

- **Description**: Average building height of geometry.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `averageBuildingHeight = 2.2`

inflowVelocityBuildingHeight

- **Description**: Inflow velocity at average building height.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `inflowVelocityBuildingHeight = 5.56`

roughnessLength

- **Description**: Roughness length z_0 of boundary layer.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `roughnessLenght = 0.004`

heightAxis

- **Description**: Axis which vertical height is measured along.
- **Required/Optional**: required
- **Type**: Named option
- **Options**: `x`, `y`, or `z`
- **Example**: `heightAxis = z`

`ZEQ4`

averageBuildingHeight

- **Description**: Average building height of geometry.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `averageBuildingHeight = 2.2`

averageBuildingWidth

- **Description**: Average building width of geometry.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `averageBuildingWidth = 1.3`

referenceHeight

- **Description**: Reference height as defined by Liu et al.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `referenceHeight = 100`

heightAxis

- **Description**: Axis which vertical height is measured along.
- **Required/Optional**: required
- **Type**: Named option
- **Options**: `x`, `y`, or `z`
- **Example**: `heightAxis = z`

2 Mesh

Specification of rectilinear finite volume mesh. Any number of segments can be specified within each grid coordinate direction.

```
Mesh {
    domainLowerBounds = ...
    domainUpperBounds = ...
    Grid x {
        Segment {
            <segment parameters>
        }
        Segment {
            <segment parameters>
        }
    }

    Grid y {
        Segment {
            <segment parameters>
        }
    }

    Grid z {
        Segment {
            <segment parameters>
        }
    }
}
```

Parameters

domainLowerBounds & domainUpperBounds

- **Description**: Lower and upper coordinate bounds of domain. This dictates the dimensions of the domain. Coordinates can be negative.
- **Required/Optional**: required
- **Type**: Vector of 3 real numbers
- **Constraints**: Must be > 0
- **Example**: `domainLowerBounds = (3, 4, 5.5)`

Segment Parameters

bounds

- **Description**: Start and end coordinates for the mesh segment
- **Required/Optional**: required
- **Type**: Vector of 2 real numbers
- **Constraints**: Bounds must be continuous between each segment and agree with overall domain dimensions.

- **Example**: `bounds = (0, 2.5)`

nCells

- **Description**: Number of cells in the mesh segment
- **Required/Optional**: required
- **Type**: Integer
- **Constraints**: Must be > 0
- **Example**: `nCells = 100`

biasFactor

- **Description**: Ratio of largest to smallest cell in the segment. A bias factor > 1 means the mesh is growing in the positive coordinate direction. If a negative value is used, the direction of growth is reversed.
- **Required/Optional**: required
- **Type**: Real number
- **Constraints**: None
- **Example**: `biasFactor = 6.5`

3 Initial Conditions

Specification of initial condition.

```
InitialConditions {
    type = ...
    <initial condition parameters>
}
```

Parameters

type

- **Description**: How the initial condition will be specified
- **Required/Optional**: required
- **Type**: Named option
- **Options**: `uniform`, `vtkFile`
- **Example**: `type = uniform`

Parameters (`type = uniform`)

The following must be specified for each field `u`, `v`, `w`, `p` on a separate line. Below is an example for the `u` field:

u

- **Description**: The (constant) initial condition value
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: None
- **Example**: `u = 2.5`

Parameters (`type = vtkFile`)

filename

- **Description**: Name and path of the legacy VTK (.vtk file extension) file to read the initial condition from. The initial condition must use the same mesh that is used in the solver.
- **Required/Optional**: Required
- **Type**: String
- **Example**: `filename = "initial/condition.vtk"`

4 Boundary Conditions

Specification of domain boundary conditions. A separate boundary condition parameter block should be specified for each domain boundary.

```
BoundaryConditions {
    +x {
        <boundary condition parameters>
    }

    -x {
        <boundary condition parameters>
    }

    +y {
        <boundary condition parameters>
    }

    -y {
        <boundary condition parameters>
    }

    +z {
        <boundary condition parameters>
    }

    -z {
        <boundary condition parameters>
    }
}
```

Parameters

The following must be specified for each field `u`, `v`, `w`, `p`. Below is an example for the `u` field:

u

- **Description**: The boundary condition applied to this field.
- **Required/Optional**: Required
- **Type**: Named option.
- **Options**:

uniform, VAL	Fixed uniform value of value VAL, where VAL is a real number
zeroGradient	Zero normal gradient.
extrapolated	Field value is extrapolated from interior using linear extrapolation in the normal direction.
profile1D, "path/filename.csv"	1D profile that is constant in the other dimension. Profile is specified from a .csv file, see below for format details.
periodic	Periodic boundary condition. Must be applied to both opposing faces on the same axis simultaneously.

- **Example**: `u  = uniform, 0`

profile1D file format:

```
y      , value
0.2    , 0.0
0.4999, 0.0
0.5001, 1
3.0    , 1
```

The first column specifies the coordinate location of each value. Its header value should be the axis that the profile is defined along, either `x`, `y`, or `z`. The second column is the value that the field will take, the header name must be `value`. Values are interpolated linearly between each value. If the domain bounds extend beyond the data specified in the file, then the last value specified in the respective direction in the profile data is used. For example in the file above, at location 4, a value of 1 will be set, and at location 0.2, a value of 0.2 will be set.

5 Solid Geometry

Specification of solid geometries within the domain to be treated through the immersed boundary method. This block and sub-blocks are optional. Multiple sub-blocks (geometry objects) can be specified and the union of them will be used. Geometries can be specified to be outside the domain bounds, only parts of the geometry within the domain bounds will contribute to calculations.

```
SolidGeometry {
    boundaryTreatment = <boundary treatment>

    Sphere {
        <sphere parameters>
    }

    Block {
        <block parameters>
    }

    FromFile {
        <geometry file parameters>
    }
}
```

boundaryTreatment

- **Description**: The method for treating solid boundaries specified by the geometries given.
- **Required/Optional**: Required
- **Type**: Named option.
- **Options**:

staircase	Uses staircase approximation for solid boundaries. This still uses the immersed boundary method mechanics, but sets distances so that it is mathematically equivalent to a staircase approximation.
directionalImmersedBoundary	Uses mass conservative direction immersed boundary method.

- **Example**: `boundaryTreatment = directionalImmersedBoundary`

`Sphere` parameters

centerPosition

- **Description**: The coordinate location in the domain of the center of the sphere.
- **Required/Optional**: Required
- **Type**: Vector of 3 real numbers.
- **Constraints**: None.
- **Example**: `centerPosition = (1, 2.5, 4)`

diameter

- **Description**: Diameter of the sphere.

- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: None.
- **Example**: `diameter = 1.3`

`Block` parameters

centerPosition

- **Description**: The coordinate location in the domain of the centroid of the block.
- **Required/Optional**: Required
- **Type**: Vector of 3 real numbers.
- **Constraints**: None.
- **Example**: `centerPosition = (1, 2.5, 4)`

dimensions

- **Description**: Dimensions in each coordinate direction (before rotation) of the block.
- **Required/Optional**: Required
- **Type**: Vector of 3 real numbers
- **Constraints**: None.
- **Example**: `dimensions = (1, 2.5, 4)`

rotation

- **Description**: Rotation of the block in each coordinate axis, performed in the order `x`, `y` `z`.
- **Required/Optional**: Required
- **Type**: Vector of 3 real numbers
- **Constraints**: None.
- **Example**: `rotation = (35, 25, 90)`

`FromFile` parameters

type

- **Description**: The file format used to specify the geometry.
- **Required/Optional**: Required
- **Type**: Word option.
- **Options**: `STL`
- **Example**: `type = STL`

filename

- **Description**: Filename for geometry file.
- **Required/Optional**: Required
- **Type**: String.
- **Constraints**: None.
- **Example**: `filename = "path/filename.stl"`

rotation

- **Description**: Rotation of the geometry in each coordinate axis, performed in the order `x`, `y` `z` about the origin in the local coordinates of geometry that is read in.
- **Required/Optional**: Optional. Defaults to no rotation, i.e. `(0, 0, 0)`.
- **Type**: Vector of 3 real numbers
- **Constraints**: None.
- **Example**: `rotation = (35, 25, 90)`

6 Solver

Specification of solver settings.

```
Solver {
    Schemes {
        <scheme parameters>
    }

    Smoother {
        <linear solver parameters>
    }

    Multigrid {
        <multigrid parameters>
    }
}
```

`Schemes` parameters

momentumInterpolation

- **Description**: Treatment of Momentum Weighted Interpolation (MWI) used for face fluxes in continuity equation.
- **Required/Optional**: Required
- **Type**: Word option.
- **Options**:

implicit	All terms are treated implicitly. This is the most robust, but results in a larger stencil.
semiExplicit	Compact gradient part of MWI is treated implicitly, while sparse gradient part is treated explicitly. This uses a smaller stencil and results in smaller memory usage, but less robustness in the solver.

.

- **Example**: `linearsation  = Picard`

advectionScheme

- **Description**: Advection scheme. First order upwind is done implicitly, with higher order schemes being applied through deferred correction.
- **Required/Optional**: Required
- **Type**: Word option.
- **Options**: `upwind` (first order upwind), `central` (second order central), `SOU` (Second Order Upwind), `QUICK` (Quadratic Upwind Interpolation for Convective Kinematics).
- **Example**: `advectionScheme = QUICK`

advectionBlendingFactor

- **Description**: Linear blending factor β between first order upwind and higher order scheme.

The blending factor is applied using the following formula

$$F = F^{\text{implicit}}_{\text{upwind}} + \beta(F^{\text{explicit}}_{\text{higher order scheme}} - F^{\text{explicit}}_{\text{upwind}}).$$

where F is the advective flux.

- **Required/Optional**: Optional, default value is 1.
- **Type**: Real number.
- **Constraints**: Should be between 0 and 1.
- **Example**: `advectionBlendingFactor = 0.75`

maxOuterIterations

- **Description**: Maximum number of outer iterations before solver terminates.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0.
- **Example**: `maxOuterIterations = 500`

maxContinuityOuterResidual

- **Description**: Maximum residual tolerance on outer iterations for continuity equation before solver stops.
- **Required/Optional**: Required.
- **Type**: Real number.
- **Constraints**: ≥ 0.
- **Example**: `maxContinuityOuterResidual = 1e-6`

maxMomentumOuterResiduals

- **Description**: Maximum residual tolerance on outer iterations for momentum equations before solver stops.
- **Required/Optional**: Required.
- **Type**: Vector of 3 real numbers. One for each momentum equation.
- **Constraints**: ≥ 0.
- **Example**: `maxMomentumOuterResiduals = (1e-6, 1e-6, 1e-6)`

The following parameters are only required if the option `transient = yes` option is set. In this case, the options above regards outer iterations refer to the iterations that are performed within each timestep to solve the implicit set of non-linear equations.

timeScheme

- **Description**: The type of discretisation used from discretising the unsteady term.
- **Required/Optional**: Required when `transient = yes`.
- **Type**: Word option.
- **Options**:

backwardsEuler	First order backwards in time.
backwardsThreeLevel	Second order backwards-in-time scheme. Requires storage of the two previous timesteps.

.

- **Example**: `timeScheme = backwardsThreeLevel`

timeStepSize

- **Description**: Size of the timestep.
- **Required/Optional**: Required when `transient = yes`.
- **Type**: Real number.
- **Constraints**: ≥ 0.
- **Example**: `timeStepSize = 0.01`

numberOfTimesteps

- **Description**: Number of timesteps to solve.
- **Required/Optional**: Required when `transient = yes`.
- **Type**: Integer.
- **Constraints**: ≥ 0.
- **Example**: `numberOfTimesteps = 150`

`Smoother` parameters

type

- **Description**: Type of smoother used for linearised equations.
- **Required/Optional**: Required.
- **Type**: Word option.
- **Options**:

nestedLineSymmetricSerial	The domain is updated in serial by sweeping planes back and forth. These planes are updated by sweeping lines back and forth, which themselves are updated by sweeping points back and forth.
domainSymmetricSerial	A single forward and backward sweep in serial is done through the domain to update each point.
domainSymmetricParallel	A single parallel forward and backward sweep is done through the domain to update each point. 3 Colour order is used to parallelise.

.

- **Example**: `type = nestedLineSymmetric`

maxIterations

- **Description**: Maximum number of linear (inner) iterations to perform on linear equations, i.e. number of iterations to perform within each outer iteration.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0.
- **Example**: `maxIterations = 3`

maxResiduals

- **Description**: Maximum residuals on all linearised equations before stopping inner iterations.

- **Required/Optional**: Required.
- **Type**: Real number.
- **Constraints**: ≥ 0.
- **Example**: `maxResiduals = 1e-3`

planeSweepDirection & lineSweepDirection

- **Description**: Plane and line sweeping direction for solution update.
- **Required/Optional**: Required.
- **Type**: Word option.
- **Options**: `+x`, `-x`, `+y`, `-y`, `+z`, `-z`.
- **Constraints**: Plane and line sweeping directions cannot be along the same axis.
- **Example**: `planeSweepingDirection = +z`, `lineSweepingDirection = -y` (must be on seperate lines)

momentumRelaxation

- **Description**: Explicit relaxation applied at the outer iterations to the momentum equations.
- **Required/Optional**: Required.
- **Type**: Vector of 3 real numbers. One for each momentum equation.
- **Constraints**: none.
- **Example**: `momentumRelaxation = (0.7, 0.7, 0.7)`

pressureRelaxation

- **Description**: Explicit relaxation applied at the outer iterations to the pressure update.
- **Required/Optional**: Required.
- **Type**: Real number.
- **Constraints**: none.
- **Example**: `pressureRelaxation = 0.7`

eddyViscosityRelaxation

- **Description**: Explicit relaxation applied at the outer iterations to the eddy viscosity field.
- **Required/Optional**: Required when a turbulence model other than `laminar` is used.
- **Type**: Real number.
- **Constraints**: none.
- **Example**: `eddyViscosityRelaxation = 0.7`

`Multigrid` parameters

cycle

- **Description**: Type of multigrid cycle to use. FMG initialisation is always used.
- **Required/Optional**: Required.
- **Type**: Word option.
- **Options**: `V`, `F`, `W`
- **Example**: `cycle = W`

maxCoarseLevels

- **Description**: Maximum number of coarsened grid levels to use. A value of zero means the

problem is solved on a single grid.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0
- **Example**: `maxCoarseLevels = 3`

preSmoothingIterations

- **Description**: Number of smoothing iterations (outer iterations) to perform before restriction at each grid level.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0
- **Example**: `preSmoothingIterations = 3`

postSmoothingIterations

- **Description**: Number of smoothing iterations (outer iterations) to perform after fine grid correction at each grid level.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0
- **Example**: `postSmoothingIterations = 3`

fineGridIterations

- **Description**: Number of smoothing iterations (outer iterations) to perform on the finest grid.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0
- **Example**: `fineGridIterations = 3`

maxCoarseGridIterations

- **Description**: Maximum number of outer iterations to perform when solving the coarsest grid.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: ≥ 0
- **Example**: `maxCoarseGridIterations = 3`

maxCoarseGridResiduals

- **Description**: Maximum residuals for all quantities for coarse grid solver.
- **Required/Optional**: Required.
- **Type**: Real number.
- **Constraints**: ≥ 0
- **Example**: `maxCoarseGridResiduals = 1e-5`

7 Parallel

Settings for running the solver in parallel.

```
Parallel {
    <model parameters>
}
```

Parameters

numberOfThreads

- **Description**: Number of threads to use.
- **Required/Optional**: required
- **Type**: Integer
- **Constraints**: Must be > 0
- **Example**: `numberOfThreads = 8`

8 Output

Specification of solver output. Specification of the Monitors block (and Probes within) is optional.

```
Output {
    <output paramters>

    Monitors {
        Probe {
            <probe parameters>
        }

        ForceCalculator {
            <probe parameters>
        }

        yPlus {
            <probe parameters>
        }
    }
}
```

Parameters

outputFormatType

- **Description**: The format/encoding to use for output files. This only affects fields, profiling, residual, and monitoring data will be in ASCII.
- **Required/Optional**: Optional
- **Type**: Word option.
- **Options**: `BINARY`, `ASCII`.
- **Example**: `linearsation  = BINARY`

residualHistoryFilename

- **Description**: Filename for file which residual history will be written to. Written in .csv format.
- **Required/Optional**: Required.
- **Type**: String
- **Constraints**: None.
- **Example**: `residualHistoryFilename = "path/residual\_history.csv"`

profilingFilename

- **Description**: Filename for file which solver profiling information will be written to.
- **Required/Optional**: Required.
- **Type**: String
- **Constraints**: None.
- **Example**: `profilingFilename = "path/profiling.dat"`

fieldOutputFilename

- **Description**: Filename for file which solution field data will be written to in legacy VTK format.
- **Required/Optional**: Required.
- **Type**: String
- **Constraints**: None.
- **Example**: `fieldOutputFilename = "path/fields.vtk"`

geometryOutputFilename

- **Description**: Filename for file which .stl file containing solid geometry is written to.
- **Required/Optional**: Optional. Not specifying this parameter will mean that geometry is not output to file.
- **Type**: String
- **Constraints**: None.
- **Example**: `geometryOutputFilename = "path/geometry.stl"`

fieldWriteInterval

- **Description**: How often to write field data to file in iterations. 1 means every iteration, 2 means every second iteration. Specifying 0 will mean field is only written at the end of the solution.
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: None.
- **Example**: `fieldWriteInterval = 2`

Probe Parameters

Specifying a probe is optional, and multiple probes can be specified. A probe simply records the value of all the fields at the given location. The value is obtained through trilinear interpolation from the surrounding cells.

filename

- **Description**: Filename for file which which probe data is written to in .csv format.
- **Required/Optional**: Required.
- **Type**: String.
- **Constraints**: None.
- **Example**: `filename = "path/probe.csv"`

location

- **Description**: Location of the probe to sample data from.
- **Required/Optional**: Required.
- **Type**: Vector of 3 real numbers.
- **Constraints**: Must be within domain bounds.
- **Example**: `location = (1, 0.5, 0.5)`

ForceCalculator Parameters

Specifying a ForceCalculator is optional. The ForceCalculator calculates the total force that the fluid exerts on all solid objects, excluding domain boundaries. If multiple solids are specified, the total force is calculated i.e. the forces all summed up.

filename

- **Description**: Filename for file which which force data is written to in .csv format.
- **Required/Optional**: Required.
- **Type**: String.
- **Constraints**: None.
- **Example**: `filename = "path/forces.csv"`

yPlusCalculator Parameters

Specifying a yPlusCalculator is optional. The yPlus calculator calculates the minimum, maximum, and average $y+$ value of all the cells that border a solid, including immersed boundaries.

filename

- **Description**: Filename for file which which yPlus data is written to in .csv format.
- **Required/Optional**: Required.
- **Type**: String.
- **Constraints**: None.
- **Example**: `filename = "path/yPlus.csv"`

9 Example Input File

```
// Example input file

Model {
    nu   = 1.57e-5
    transient = no
    TurbulenceModel {
        model                         = ZEQ3
        heightAxis                    = z
        averageBuildingHeight         = 100
        inflowVelocityBuildingHeight = 1
        roughnessLength               = 0.0004
    }
}

Mesh {
    domainLowerBounds = (-500  , -350, 0)
    domainUpperBounds = (1000   , 350, 400)

    Grid x {
        Segment {
            bounds     = (-500, -50)
            nCells     = 45
            biasFactor = -3.5
        }

        Segment {
            bounds     = (-50, 50)
            nCells     = 25
            biasFactor = 1
        }

        Segment {
            bounds     = (50, 1000)
            nCells     = 95
            biasFactor = 4.5
        }
    }


    Grid y {
        Segment {
            bounds     = (-350, -50)
            nCells     = 30
            biasFactor = -3.5
        }

        Segment {
            bounds     = (-50, 50)
            nCells     = 25
            biasFactor = 1
        }

        Segment {
            bounds     = (50, 350)
            nCells     = 30
            biasFactor = 3.5
```

```
        }
    }


    Grid z {
        Segment {
            bounds     = (0, 100)
            nCells     = 25
            biasFactor = 1
        }

        Segment {
            bounds     = (100, 400)
            nCells     = 35
            biasFactor = 4.75
        }
    }
}


InitialConditions
{
    type = uniform
    u = 1
    v = 0
    w = 0
    p = 0
}


BoundaryConditions
{
    // Outflow
    +x {
        u = zeroGradient
        v = zeroGradient
        w = zeroGradient
        p = uniform, 0
    }

    // Inflow
    -x {
        u = uniform, 1
        v = uniform, 0
        w = uniform, 0
        p = zeroGradient
    }

    // Wall
    +y {
        u = uniform, 0
        v = uniform, 0
        w = uniform, 0
        p = zeroGradient
    }

    // Wall
    -y {
        u = uniform, 0
```

```
        v = uniform, 0
        w = uniform, 0
        p = zeroGradient
    }

    // Symmetry
    +z {
        u = zeroGradient
        v = zeroGradient
        w = uniform, 0
        p = zeroGradient
    }

    // Wall
    -z {
        u = uniform, 0
        v = uniform, 0
        w = uniform, 0
        p = zeroGradient
    }
}


SolidGeometry {
    boundaryTreatment = directionalImmersedBoundary
    FromFile {
        type = STL
        filename = "geometry.stl"
        rotation = (0, 0, -10)
    }

}


Solver {

    Schemes {

        linearisation           = Picard
        momentumInterpolation   = implicit
        advectionScheme         = upwind
        advectionBlendingFactor = 1.0
        faceInterpolationScheme = weightedLinear

        maxOuterIterations = 11
        maxContinuityOuterResidual = 1e-12
        maxMomentumOuterResiduals  = (1e-12, 1e-12, 1e-12)

    }

    Smoother {

        type = domainSymmetricParallel
        maxIterations = 3
        maxResiduals = 1e-5

        planeSweepDirection = +z
        lineSweepDirection = +x

```

```
        eddyViscosityRelaxation = 1
        momentumRelaxation      = (0.85, 0.85, 0.85)
        pressureRelaxation      = 1

    }

    Multigrid {
        cycle = F
        maxCoarseLevels = 3

        preSmoothingIterations = 3
        postSmoothingIterations = 3
        fineGridIterations   = 3

        maxCoarseGridIterations = 50
        maxCoarseGridResiduals  = 1e-10

    }

}

Parallel {
    numberOfThreads = 8
}

Output {

    outputFormatType        = BINARY
    residualHistoryFilename = "../../output/residual_history.csv"
    profilingFilename       = "../../output/profiling.dat"
    fieldOutputFilename     = "../../output/fields.vtk"
    geometryOutputFilename  = "../../output/geometry.stl"
    fieldWriteInterval      = 0

    Monitors {
        ForceCalculator {
            filename = "../../output/forces.csv"
        }

        yPlus {
            filename = "../../output/yPlus.csv"
        }
    }

}
```

Part II

CAMIRA-PLUME

10 Model

Physical modelling parameters and time-stepping configuration for the Lagrangian particle dispersion simulation.

```
Model {
    <model parameters>
}
```

Parameters

D

- **Description**: Molecular diffusion coefficient D of the scalar quantity being dispersed.
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: Must be ≥ 0
- **Example**: `D = 1.5e-5`

turbulentSchmidtNumber

- **Description**: Turbulent Schmidt number Sc_t, used to relate the turbulent eddy diffusivity to the eddy viscosity from the flow field. The turbulent eddy diffusivity is computed as $D_t = \nu_t / Sc_t$.
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `turbulentSchmidtNumber = 0.7`

numberOfTimeSteps

- **Description**: Total number of timesteps to simulate.
- **Required/Optional**: Required
- **Type**: Integer
- **Constraints**: Must be > 0
- **Example**: `numberOfTimeSteps = 5000`

timeStepSize

- **Description**: Size of each timestep Δt.
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: Must be > 0

- **Example**: `timeStepSize = 0.05`

particleSplitTimeStepInterval

- **Description**: Interval in timesteps at which particle splitting is performed. Particle splitting subdivides particles to maintain numerical accuracy as the plume disperses.
- **Required/Optional**: Required
- **Type**: Integer
- **Constraints**: Must be > 0
- **Example**: `particleSplitTimeStepInterval = 50`

maxNumberOfParticleSplits

- **Description**: Maximum number of times any individual particle may be split over the course of the simulation.
- **Required/Optional**: Required
- **Type**: Integer
- **Constraints**: Must be ≥ 0. A value of 0 disables splitting entirely.
- **Example**: `maxNumberOfParticleSplits = 4`

initialParticlesPerUnitMass

- **Description**: Number of computational particles used to represent one unit of released mass at the time of emission. Controls the initial resolution of the particle representation.
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `initialParticlesPerUnitMass = 100`

11 Velocity Field

Specification of the velocity field used to advect particles. CAMIRA-PLUME reads a pre-computed velocity field from a legacy VTK file, typically produced by a prior CAMIRA-FLOW simulation. It will use the mesh from this file for the final concentration fields in the Eulerian frame.

```
VelocityField {
    <velocity field parameters>
}
```

Parameters

velocityFieldFilename

- **Description**: Path to the legacy VTK (`.vtk`) file containing the velocity field. The mesh in this file must match the mesh on which particles are being dispersed.
- **Required/Optional**: Required
- **Type**: String
- **Constraints**: None.
- **Example**: `velocityFieldFilename = "path/flow_fields.vtk"`

12 Solid Geometry

Specification of solid geometry within the domain, used for particle–wall collision handling. This block is optional. If omitted, no solid surfaces are present and all boundary interactions are governed solely by the domain boundary conditions.

```
SolidGeometry {
    <solid geometry parameters>
}
```

Parameters

filename

- **Description**: Path to an STL file describing the solid geometry.
- **Required/Optional**: Required if `SolidGeometry` block is present.
- **Type**: String
- **Constraints**: None.
- **Example**: `filename = "path/geometry.stl"`

13 Sources

Specification of particle release sources. Any number of sources can be specified, and multiple source types can be combined in the same simulation.

```
Sources {
    ContinuousReleasePoint {
        <continuous release parameters>
    }

    InstantaneousReleasePoint {
        <instantaneous release parameters>
    }
}
```

`ContinuousReleasePoint` parameters

A continuous release point emits particles at a steady mass flow rate for the duration of the simulation. Multiple `ContinuousReleasePoint` blocks can be specified.

location

- **Description**: Coordinate location of the release point.
- **Required/Optional**: Required
- **Type**: Vector of 3 real numbers
- **Constraints**: Should be within the domain bounds.
- **Example**: `location = (10, 0, 15)`

massFlowRate

- **Description**: Mass flow rate at which the scalar quantity is released.
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `massFlowRate = 0.01`

`InstantaneousReleasePoint` parameters

An instantaneous release point emits a fixed total mass of particles at a single point at the start of the simulation ($t = 0$). Multiple `InstantaneousReleasePoint` blocks can be specified.

location

- **Description**: Coordinate location of the release point.
- **Required/Optional**: Required
- **Type**: Vector of 3 real numbers
- **Constraints**: Should be within the domain bounds.
- **Example**: `location = (10, 0, 15)`

totalMass

- **Description**: Total mass of scalar quantity released instantaneously.
- **Required/Optional**: Required
- **Type**: Real number
- **Constraints**: Must be > 0
- **Example**: `totalMass = 1.0`

14 Boundary Conditions

Specification of domain boundary conditions for the particle concentration field ϕ. A boundary condition must be specified for each of the six domain faces.

```
BoundaryConditions {
    +x {
        <boundary condition parameters>
    }

    -x {
        <boundary condition parameters>
    }

    +y {
        <boundary condition parameters>
    }

    -y {
        <boundary condition parameters>
    }

    +z {
        <boundary condition parameters>
    }

    -z {
        <boundary condition parameters>
    }
}
```

Parameters

The following parameter must be specified for the concentration field `phi` within each boundary patch block.

phi

- **Description**: The boundary condition applied to the particle concentration field at this face.
- **Required/Optional**: Required
- **Type**: Named option.
- **Options**:

outflow	Particles that reach this boundary are removed from the simulation. Suitable for outflow and open boundaries.
reflection	Particles that reach this boundary are reflected (specular reflection) back into the domain. Suitable for solid walls and symmetry planes.
periodic	Particles that exit through this boundary re-enter through the opposing face on the same axis. Must be applied to both opposing faces simultaneously.

- **Example**: `phi = outflow`

15 Parallel

Settings for running the solver in parallel.

```
Parallel {
    <parallel parameters>
}
```

Parameters

numberOfThreads

- **Description**: Number of threads to use.
- **Required/Optional**: Required
- **Type**: Integer
- **Constraints**: Must be > 0
- **Example**: `numberOfThreads = 8`

16 Output

Specification of solver output. The `TimeAveragedConcentration` sub-block is optional and multiple instances can be specified.

```
Output {
    <output parameters>

    TimeAveragedConcentration {
        <time-averaged concentration parameters>
    }

    TimeAveragedConcentration {
        <time-averaged concentration parameters>
    }
}
```

Parameters

outputFormatType

- **Description**: The encoding format for field output files. Profiling and monitoring data are always written in ASCII regardless of this setting.
- **Required/Optional**: Optional. Defaults to `BINARY`.
- **Type**: Word option.
- **Options**: `BINARY`, `ASCII`.
- **Example**: `outputFormatType = BINARY`

profilingFilename

- **Description**: Filename for the file to which solver profiling information will be written.
- **Required/Optional**: Required.
- **Type**: String
- **Constraints**: None.
- **Example**: `profilingFilename = "path/profiling.dat"`

fieldOutputFilename

- **Description**: Filename for the legacy VTK file to which instantaneous particle concentration field data will be written.
- **Required/Optional**: Required.
- **Type**: String
- **Constraints**: None.
- **Example**: `fieldOutputFilename = "path/concentration.vtk"`

fieldWriteInterval

- **Description**: How often to write the instantaneous concentration field to file, in timesteps. A value of 1 writes every timestep, 2 writes every second timestep, and so on. Specifying 0 means the field is only written at the end of the simulation.
- **Required/Optional**: Required.

- **Type**: Integer.
- **Constraints**: Must be ≥ 0.
- **Example**: `fieldWriteInterval = 10`

`TimeAveragedConcentration` parameters

Computes and writes a time-averaged concentration field by averaging the instantaneous field over a specified range of timesteps. Multiple `TimeAveragedConcentration` blocks can be specified to produce averages over different time windows.

filename

- **Description**: Filename for the legacy VTK file to which the time-averaged concentration field will be written.
- **Required/Optional**: Required.
- **Type**: String.
- **Constraints**: None.
- **Example**: `filename = "path/time_avg_concentration.vtk"`

startTimeStep

- **Description**: The timestep at which the time-averaging window begins (inclusive).
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: Must be ≥ 0 and less than `endTimeStep`.
- **Example**: `startTimeStep = 1000`

endTimeStep

- **Description**: The timestep at which the time-averaging window ends (inclusive).
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: Must be greater than `startTimeStep`.
- **Example**: `endTimeStep = 5000`

timeStepInterval

- **Description**: Interval in timesteps at which samples are taken within the averaging window. Must divide exactly into the total window length (`endTimeStep` − `startTimeStep`).
- **Required/Optional**: Required.
- **Type**: Integer.
- **Constraints**: Must be ≥ 1 and must divide evenly into (`endTimeStep` − `startTimeStep`).
- **Example**: `timeStepInterval = 10`

17 Example Input File

```
// Example CAMIRA-PLUME input file

Model {
    D                            = 1.5e-5
    turbulentSchmidtNumber       = 0.7
    numberOfTimeSteps            = 5000
    timeStepSize                 = 0.05
    particleSplitTimeStepInterval = 50
    maxNumberOfParticleSplits    = 4
    initialParticlesPerUnitMass  = 100
}

VelocityField {
    velocityFieldFilename = "../../flow/output/fields.vtk"
}

SolidGeometry {
    filename = "geometry.stl"
}

Sources {
    ContinuousReleasePoint {
        location     = (10, 0, 15)
        massFlowRate = 0.01
    }

    InstantaneousReleasePoint {
        location  = (-5, 0, 10)
        totalMass = 1.0
    }
}

BoundaryConditions {
    // Outflow
    +x {
        phi = outflow
    }

    // Inflow
    -x {
        phi = outflow
    }

    // Lateral
    +y {
        phi = outflow
    }

    -y {
        phi = outflow
    }

    // Top
    +z {
        phi = outflow
    }
```

```
57
58      // Ground
59      -z {
60          phi = reflection
61      }
62  }
63
64  Parallel {
65      numberOfThreads = 8
66  }
67
68  Output {
69      outputFormatType    = BINARY
70      profilingFilename   = "../../output/profiling.dat"
71      fieldOutputFilename = "../../output/concentration.vtk"
72      fieldWriteInterval  = 50
73
74      TimeAveragedConcentration {
75          filename         = "../../output/time_avg_full.vtk"
76          startTimeStep    = 1000
77          endTimeStep      = 5000
78          timeStepInterval = 10
79      }
80
81      TimeAveragedConcentration {
82          filename         = "../../output/time_avg_late.vtk"
83          startTimeStep    = 3000
84          endTimeStep      = 5000
85          timeStepInterval = 10
86      }
87  }
```